



IMDB MOVIE DATA ANALYSIS

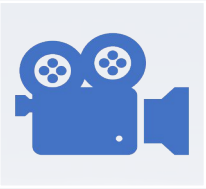
**PANDAS AND NUMPY
LIBRARIES**

**SCHOOL OF DATA SCIENCE
AND FORECASTING**
DAVV

BY – ARPIT DUBEY



DATASET DESCRIPTION



- CSV format.
- Movie Title, Year, Country. Genre, Total Review, Language, Votes
- IMDb rating & duration.

B8

2000

	A	B	C	D	E	F	G	H	I
1	Movie_title	Year	Genre	Language	Country	Imdb_rating	Total_reviews	Voted_users	Duration_minutes
2	The Host	2018	Comedy	Korean	South Korea	7	363	238760	94
3	Lady Vengeance	2004	Crime	Korean	South Korea	9	202	399371	104
4	Fateless	1997	Drama	Hungarian	Hungary	7.4	73	307503	115
5	Princess Mononoke	2010	Adventure	Japanese	Japan	4.6	174	447341	154
6	Steamboy	2008	Action	Japanese	Japan	9.2	105	314742	130
7	Akira	2012	Action	Japanese	Japan	7.5	150	146823	94
8	Godzilla 2000	2000	Action	Japanese	Japan	5.8	107	122913	147
9	Tango	2000	Drama	Spanish	Spain	4.8	35	263170	100
10	Kabhi Alvida Naa Kehna	2013	Drama	Hindi	India	8.4	20	331404	92
11	Kites	2013	Action	English	India	7.4	41	48337	165
12	Red Cliff	1992	Action	Mandarin	China	6.9	160	261050	98
13	The Legend of Suriyothai	2011	Action	Thai	Thailand	8.9	31	274180	113
14	The Messenger: The Story of Joan of Arc	1991	Adventure	English	France	8.3	111	29041	157
15	Pirates of the Caribbean: At World's End	2013	Action	English	USA	4.8	302	79583	147
16	Ong-bak 2	2019	Action	Thai	Thailand	5.7	110	444743	131
17	John Carter	1991	Action	English	USA	5.4	462	35031	106
18	Tangled	2010	Adventure	English	USA	8.1	324	271356	108

TOOLS USED

Python

Pandas

NumPy

Jupyter
Notebook

Q – Data loading through CSV file

- `df = pd.read_csv(r"C:\Users\win10\Documents\IMDB_Movies_Dataset.csv")`

```
] import pandas as pd  
import numpy as np
```

```
] #importing file  
df = pd.read_csv(r"C:\Users\win10\Documents\IMDB_Movies_Dataset.csv")
```

Q – Data preview

- Data Info
- Data Dimension

•[3]:

```
#total data in dataset
```

```
df.info()
```

```
df.shape[0]
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 500 entries, 0 to 499
```

```
Data columns (total 9 columns):
```

#	Column	Non-Null Count	Dtype
0	Movie_title	500 non-null	object
1	Year	499 non-null	float64
2	Genre	499 non-null	object
3	Language	498 non-null	object
4	Country	499 non-null	object
5	Imdb_rating	500 non-null	float64
6	Total_reviews	495 non-null	float64
7	Voted_users	496 non-null	float64
8	Duration_minutes	498 non-null	float64

```
dtypes: float64(5), object(4)
```

```
memory usage: 35.3+ KB
```

[3]: 500

■ Top Five data

```
[7]: # Q -- Top five value of data
```

```
df.head()
```

```
[7]:
```

	Movie_title	Year	Genre	Language	Country	Imdb_rating	Total_reviews	Votes	Duration_minutes
0	The Host	2018.0	Comedy	Korean	South Korea	7.0	363.0	238760.0	94.0
1	Lady Vengeance	2004.0	Crime	Korean	South Korea	9.0	202.0	399371.0	104.0
2	Fateless	1997.0	Drama	Hungarian	Hungary	7.4	73.0	307503.0	115.0
3	Princess Mononoke	2010.0	Adventure	Japanese	Japan	4.6	174.0	447341.0	154.0
4	Steamboy	2008.0	Action	Japanese	Japan	9.2	105.0	314742.0	130.0

■ Bottom Five data

```
[8]: # Q -- bottom Five data
```

```
df.tail()
```

```
[8]:
```

	Movie_title	Year	Genre	Language	Country	Imdb_rating	Total_reviews	Votes	Duration_minutes
495	Titan A.E.	1997.0	Action	English	USA	8.2	NaN	326750.0	91.0
496	Unbreakable	2017.0	Drama	English	USA	8.6	294.0	463683.0	NaN
497	Vertical Limit	2015.0	Action	English	USA	8.2	134.0	59682.0	81.0
498	X-Men	1997.0	Action	English	USA	7.4	290.0	487016.0	92.0
499	Monkeybone	2017.0	Animation	English	USA	4.7	74.0	349807.0	132.0

Q – Data Cleaning

- Checking Null Values
- Renaming

```
[4]: #Q -- Renaming column using inplace which will change in Original  
  
df.rename(columns={"Voted_users":"Votes"}, inplace=True)  
  
# using inplace which will change in Original
```

```
[5]: # Q -- Count and null values  
  
df.isnull().sum()
```

```
[5]: Movie_title      0  
Year                1  
Genre              1  
Language           2  
Country            1  
Imdb_rating        0  
Total_reviews      5  
Votes              4  
Duration_minutes   2  
dtype: int64
```


■ Removing Duplicates

```
[17]: #@removing the duplicate data
```

```
df.drop_duplicates()
```

```
[17]:
```

	Movie_title	Year	Genre	Country	Language	Total_reviews	Imdb_rating	Votes	Duration_minutes
0	Durandhar	2025.0	Action	India	Hindi	45201	9.3	578454.0	205.0
1	The Host	2018.0	Comedy	South Korea	Korean	363.0	7.0	238760.0	94.0
2	Lady Vengeance	2004.0	Crime	South Korea	Korean	202.0	9.0	399371.0	104.0
3	Fateless	1997.0	Drama	Hungary	Hungarian	73.0	7.4	307503.0	115.0
4	Princess Mononoke	2010.0	Adventure	Japan	Japanese	174.0	4.6	447341.0	154.0
...	...	...	...	...	...	...	...	...	...
496	Titan A.E.	1997.0	Action	USA	English	NaN	8.2	326750.0	91.0
497	Unbreakable	2017.0	Drama	USA	English	294.0	8.6	463683.0	NaN
498	Vertical Limit	2015.0	Action	USA	English	134.0	8.2	59682.0	81.0
499	X-Men	1997.0	Action	USA	English	290.0	7.4	487016.0	92.0
500	Monkeybone	2017.0	Animation	USA	English	74.0	4.7	349807.0	132.0

496 rows × 9 columns

Q – Statistical Analysis

- Gives the Statistical data like Mean, min, std, etc

```
[6]: # Q -- statistical analysis  
df.describe()  
  
#integer float values deta hai  
|
```

```
[6]:
```

	Year	Imdb_rating	Total_reviews	Votes	Duration_minutes
count	499.000000	500.000000	495.000000	496.000000	498.000000
mean	2006.819639	6.805600	264.884848	257278.643145	126.299197
std	10.039857	1.625168	155.492195	137543.298516	29.299000
min	1990.000000	4.000000	8.000000	4226.000000	80.000000
25%	1998.000000	5.300000	153.500000	144782.750000	100.000000
50%	2008.000000	6.900000	231.000000	260833.000000	123.500000
75%	2015.000000	8.200000	351.500000	373370.500000	152.000000
max	2023.000000	9.500000	813.000000	499615.000000	179.000000

Q – NumPy Mean Function

- Average Duration of all Movies
- Reviews Analytics

```
[9]: #NUMPY
# Q -- What is the average duration of all movies?

np.mean(df['Duration_minutes'])

# Statistical Aggregation (Mean) , NumPy mean function
```

```
[9]: np.float64(126.29919678714859)
```

```
[10]: # Q -- Distribution of ratings using NumPy

ratings = df['Total_reviews'].dropna().values

print("Mean:", np.mean(ratings))
print("Median:", np.median(ratings))
print("Std Dev:", np.std(ratings))
```

```
Mean: 264.8848484848485
```

```
Median: 231.0
```

```
Std Dev: 155.33505289959663
```

Q – Data Filtering

■ Multiple Condition Label Filtering

```
[11]: # Q -- label based data selection
# as it will show copy do not change the original

df.loc[(df['Year'] > 2000) & (df['Imdb_rating'] > 9.0)].sort_values(by='Year', ascending=False).head(5)
```

```
[11]:
```

	Movie_title	Year	Genre	Language	Country	Imdb_rating	Total_reviews	Votes	Duration_minutes
49	Iron Man 2	2023.0	Action	English	USA	9.1	453.0	181502.0	133.0
224	Speed Racer	2022.0	Action	English	USA	9.2	284.0	20475.0	131.0
268	The Princess and the Frog	2022.0	Animation	English	USA	9.1	228.0	45238.0	104.0
251	Ghost Rider	2021.0	Action	English	USA	9.2	276.0	102020.0	159.0
481	The Adventures of Rocky & Bullwinkle	2020.0	Adventure	English	USA	9.4	49.0	421314.0	145.0

■ Group by

```
[4]: # Q -- Longest Movie Duration by Each Country in descending Order

Longmov = df.loc[
    df.groupby('Country')['Duration_minutes'].idxmax(),
    ['Country', 'Movie_title', 'Duration_minutes']].sort_values(by='Duration_minutes', ascending=False)

Longmov = Longmov.reset_index(drop=True)

print(Longmov)
```

	Country	Movie_title	Duration_minutes
0	USA	Waterworld	179.0
1	UK	Harry Potter and the Chamber of Secrets	177.0
2	Germany	Spy Game	172.0
3	France	Les couloirs du temps: Les visiteurs II	165.0
4	India	Kites	165.0
5	Australia	Days of Our life	163.0
6	China	The Flowers of War	159.0
7	Japan	Princess Mononoke	154.0
8	Canada	X-Men: The Last Stand	153.0
9	New Zealand	King Kong	150.0
10	Thailand	The Protector	140.0
11	Hungary	Fateless	115.0
12	South Korea	Lady Vengeance	104.0
13	Spain	Tango	100.0
14	Czech Republic	I Served the King of England	88.0
15	Philliphines	Town & Country	81.0

Q – Show Cinema Boom in Decades

- Decade column will be added, but we will remove it after using drop for further operation
- If we use Year, results will be too detailed (year-by-year).

```
2]: # Q -- Which decade had the most movies?

df["Decade"] = (df["Year"]//10)*10    #2004 // 10 = 200 , 200*10 = 2000 Year Decade
print(df["Decade"].value_counts().sort_index())

#decade column will added
#Shows cinema boom decades.
```

```
Decade
1990.0    144
2000.0    132
2010.0    161
2020.0     62
Name: count, dtype: int64
```


Q – New Row addition to dataset

```
5]: # Q --- Concatenat, Row addition

new_row = pd.DataFrame({"Movie_title": 'Durandhar',
                        "Year": 2025,
                        "Genre": 'Action',
                        "Country": 'India',
                        "Language": 'Hindi',
                        "Total_reviews": '45201',
                        "Imdb_rating": 9.3,
                        "Votes": 578454,
                        "Duration_minutes": 205},
                        index=[0])

df = pd.concat([new_row, df]).reset_index(drop=True)
df.head(10)
```

```
5]:
```

	Movie_title	Year	Genre	Country	Language	Total_reviews	Imdb_rating	Votes	Duration_minutes
0	Durandhar	2025.0	Action	India	Hindi	45201	9.3	578454.0	205.0
1	The Host	2018.0	Comedy	South Korea	Korean	363.0	7.0	238760.0	94.0
2	Lady Vengeance	2004.0	Crime	South Korea	Korean	202.0	9.0	399371.0	104.0
3	Fateless	1997.0	Drama	Hungary	Hungarian	73.0	7.4	307503.0	115.0
4	Princess Mononoke	2010.0	Adventure	Japan	Japanese	174.0	4.6	447341.0	154.0
5	Steamboy	2008.0	Action	Japan	Japanese	105.0	9.2	314742.0	130.0
6	Akira	2012.0	Action	Japan	Japanese	150.0	7.5	146823.0	94.0
7	Godzilla 2000	2000.0	Action	Japan	Japanese	107.0	5.8	122913.0	147.0
8	Tango	2000.0	Drama	Spain	Spanish	35.0	4.8	263170.0	100.0
9	Kabhi Alvida Naa Kehna	2013.0	Drama	India	Hindi	20.0	8.4	331404.0	92.0

Print(“Thank You”)

**THANK
YOU**